



PRACTICAL – 3

AIM: Implementation of max-heap sort algorithm.

Max heap sort Algorithm:

Max Heap sort Algorithm

Input: Array elements
Output: Sorted

Step-1: start
Step-2: for $i \in n/2$ to 1, do
Step-3: largest $\leftarrow i$
Step-4: $l \leftarrow 2 * i$
Step-5: $r \leftarrow 2 * i + 1$
Step-6: if $l \leq n$ and $arr[l] > arr[largest]$ then
do largest $\leftarrow l$
Step-7: if $r \leq n$ and $arr[r] > arr[largest]$, then
do largest $\leftarrow r$
Step-8: if largest $\neq i$ then do
Step-9: swap $arr[largest]$ and $arr[i]$
Step-10: for $i \leftarrow size, i \geq 1$
Step-11: swap $arr[1]$ and $arr[i]$
Step-12: $i \leftarrow i - 1$ and
Step-13: max Heapify ($arr, i, 1, 1$)
Step-14: stop.

CODE:

```
#include<iostream> using
namespace std; void heapify(int arr[],
int n, int i)
{ int largest = i; int left = 2 * i + 1; int
right = 2 * i + 2; if (left < n &&
arr[left] > arr[largest]) largest = left;
```

```
if(right<n&&arr[right]>arr[largest])
largest = right; if (largest != i)
{ swap(arr[i],
arr[largest]); heapify(arr,
n, largest);
} } voidheapSort(int arr[],
int n)
{
// Build Max Heap
for(inti=n/2-1;i>=0;i--)
heapify(arr, n, i); // Heap Sort
for (int i = n - 1; i >0; i--)
{ swap(arr[0],
arr[i]); heapify(arr,
i, 0);
} } int main() { intn; int
arr[100]; cout << "Enter size of
array: "; cin >>n; cout <<
"Enter array elements: "; for(int
i = 0; i < n; i++) cin >> arr[i];
heapSort(arr, n); cout <<
"Sorted array: "; for(int i = 0;i
< n; i++) cout << arr[i] << " ";
return0;
}
```

OUTPUT:

```
Output
Enter size of array: 6
Enter array elements: 3 5 8 12 2 9
Sorted array: 2 3 5 8 9 12

=== Code Execution Successful ===
```



TIME COMPLEXITY:

Time complexity (Best, Average, worst)

for $(i = n/2; i > 1; i--) \rightarrow n$

max heapify (arr, n, i) $\rightarrow \log n$

for $(i = n; i > 1; i--) \rightarrow n$

swap (arr[1], arr[i]) $\rightarrow 1$

max heapify (arr, i-1, 1) $\rightarrow \log n$

$$T(n) = n * \log n * n * 1 * \log n$$

$$= n \log n + n \log n$$

$$T(n) = O(n \log n)$$

for Max heap sort:

$$\text{Best} = \text{Average} = \text{worst} = O(n \log n)$$

CONCLUSION:

The Max-Heap Sort algorithm was successfully implemented and tested. It sorts the elements by creating a max-heap and repeatedly placing the largest element in its correct position. It is efficient for large amounts of data and has a time complexity of $O(n \log n)$.